Контрольная работа № 1. Вариант 1 — демоверсия

Время: 60 минут. Максимум: 10 баллов.

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Фамилия, имя: _____ Группа: _____

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются, но не запрещаются и приветствуются, если понимаете что ваше решение немного запутанное. За частично верное решение начисляются баллы.

Рекомендуемое время: 10 + 10 + 15 + 20 минут на задачи и 5 минут на проверку.

Задача 1. Максимальное число

2 балла · 10 минут

Напишите функцию `largest_number(number)`, которая принимает целое неотрицательное число и возвращает наибольшее целое число, которое можно составить из его цифр.

Нужно использовать все цифры исходного числа, каждую — ровно столько раз, сколько она встречается. Преобразовывать число или отдельные цифры в строки нельзя.

```
def largest_number(number):
    ...
```

Примеры:

```
assert largest_number(31042) == 43210
assert largest_number(9090) == 9900
assert largest_number(7) == 7
assert largest_number(0) == 0
```

Задача 2. Самое длинное слово

2 балла · 10 минут

Напишите функцию `longest_word(text)`, которая возвращает самое длинное слово в строке. Если слов максимальной длины несколько, верните первое из них по порядку в исходной строке.

Строка содержит только буквы и пробелы. Между словами может быть несколько пробелов; пробелы также могут быть в начале и конце строки. Регистр букв нужно сохранить. Если слов нет, верните пустую строку.

```
def longest_word(text):
    ...
```

Примеры:

```
assert longest_word("Мы изучаем Python") == "изучаем"
assert longest_word("  кот   кит  дом ") == "кот"
assert longest_word("Привет") == "Привет"
assert longest_word("   ") == ""
assert longest_word("") == ""
```

Задача 3. Первые подходящие значения

2,5 балла · 15 минут

Напишите функцию `first_even_above(values, threshold, k)`.

`values` — список целых чисел, `threshold` — целое число, `k` — целое неотрицательное число.

Верните новый список из первых `k` элементов `values`, которые одновременно удовлетворяют двум условиям:

- число чётное;
- число строго больше `threshold`.

Сохраните исходный порядок и повторы. Если подходящих элементов меньше `k`, верните все подходящие. При `k == 0` верните пустой список.

Как только набрано `k` подходящих элементов, просмотр исходного списка должен прекратиться. При `k == 0` просматривать элементы не нужно.

```
def first_even_above(values, threshold, k):
    ...
```

Примеры:

```
assert first_even_above([7, 4, 8, 8, 3, 10], 4, 2) == [8, 8]
assert first_even_above([2, 9, 6, 4], 2, 5) == [6, 4]
assert first_even_above([-6, -3, -2, 0, 4], -4, 2) == [-2, 0]
assert first_even_above([2, 4, 6], 0, 0) == []
assert first_even_above([], 0, 3) == []
```

Задача 4. Сколько покупателей у товара

3,5 балла · 20 минут

Напишите функцию `buyers_per_product(purchases)`.

`purchases` — список кортежей `(покупатель, товар)`. Оба элемента кортежа — строки. Одна и та же пара может встречаться несколько раз: это означает, что человек покупал этот товар несколько раз.

Верните словарь, в котором ключ — название товара, а значение — число **разных покупателей**, купивших этот товар. Повторные покупки одного и того же товара одним человеком не должны увеличивать это число. Один человек может покупать разные товары.

В результат должны войти все товары из исходного списка. Порядок ключей в результате не важен. Для пустого списка верните пустой словарь. Имена и названия сравниваются точно, с учётом регистра; очищать или преобразовывать строки не нужно.

Требование к эффективности: среднее время работы `O(n)`, где `n` — число записей в `purchases`. Для этой оценки сравнение и поиск/вставку строк в словарь считайте операциями постоянной стоимости. Объяснять сложность письменно не нужно.

```
def buyers_per_product(purchases):
    ...
```

Примеры:

```
assert buyers_per_product([
    ("Аня", "чай"),
    ("Борис", "хлеб"),
    ("Аня", "чай"),
    ("Вера", "чай"),
    ("Аня", "хлеб"),
    ("Борис", "хлеб"),
]) == {"чай": 2, "хлеб": 2}

assert buyers_per_product([("Аня", "чай"), ("Аня", "чай")]) == {"чай": 1}
assert buyers_per_product([("Аня", "чай"), ("Аня", "хлеб")]) == {"чай": 1,
"хлеб": 1}
assert buyers_per_product([]) == {}
```